



Tecnológico de Monterrey

Campus Santa Fe

Informe Reto de Movilidad Urbana

Santiago Coronado Hernández
Luis Emilio Veledíaz Flores

Octavio Navarro Hinojosa
Gilberto Echeverría Furió

Modelación de sistemas multiagentes con gráficas computacionales
Gpo. 302

4 de Diciembre del 2025

Problema a Resolver

Cifras del Instituto Nacional de Estadística y Geografía (INEGI), revelan que entre mayo de 2020 y mayo de 2025, el número total de automóviles en circulación pasó de 34.36 a 38.85 millones. De estos, 38.1 millones son de uso particular. Este incremento representa un crecimiento absoluto de 4 millones de unidades, con un promedio de 898 mil 696 automóviles añadidos por año.

El reto consiste en proponer una solución al problema de movilidad urbana en México, mediante la creación de un modelo que simula el comportamiento en una ciudad con vehículos; con la finalidad de implementar métodos, algoritmos y conocimientos que permitan que la simulación sea lo más eficiente posible. El modelo implementa agentes que operan dentro de una cuadrícula que parece un ambiente urbano. Los agentes perciben su entorno local y se van ajustando dinámicamente sus movimientos para maximizar el flujo de todos los vehículos.

Propuesta de Solución

El reto se aborda mediante el diseño e implementación de una simulación basada en Sistemas Multi Agentes. La solución propone un modelo de inteligencia autónoma, que no depende de un control o revisión constante de alguien ajeno a la simulación. La solución incluye la creación de un entorno que representa las calles, y dentro de este entorno se despliegan agente de tipo vehículo con comportamientos como:

- Tener destinos aleatorios.
- Reaccionar a los semáforos y la presencia de otros vehículos.
- Buscar la ruta más eficaz en tiempo real dependiendo de las condiciones de tráfico actuales.

La propuesta está realizada mediante una arquitectura de subsunción, donde ciertas reglas tienen prioridad sobre otras, por lo que resulta en un comportamiento general de optimización. Este modelo nos permite probar algoritmos para generar un entorno controlado en cierta medida. Nuestro modelo tiene una temática centrada en el universo de TRON, juntamos modelos de todas las películas para lograr que pareciera una simulación de una ciudad, pero dentro del universo de la temática. Nuestra simulación cuenta con:

- Motos que simulan ser los vehículos
- Semáforos de acuerdo a la temática
- Edificios de acuerdo a la temática

A continuación se muestran algunas imágenes tanto de los modelo, como de el mapa en 3D que se utilizó usando gráficas computacionales:

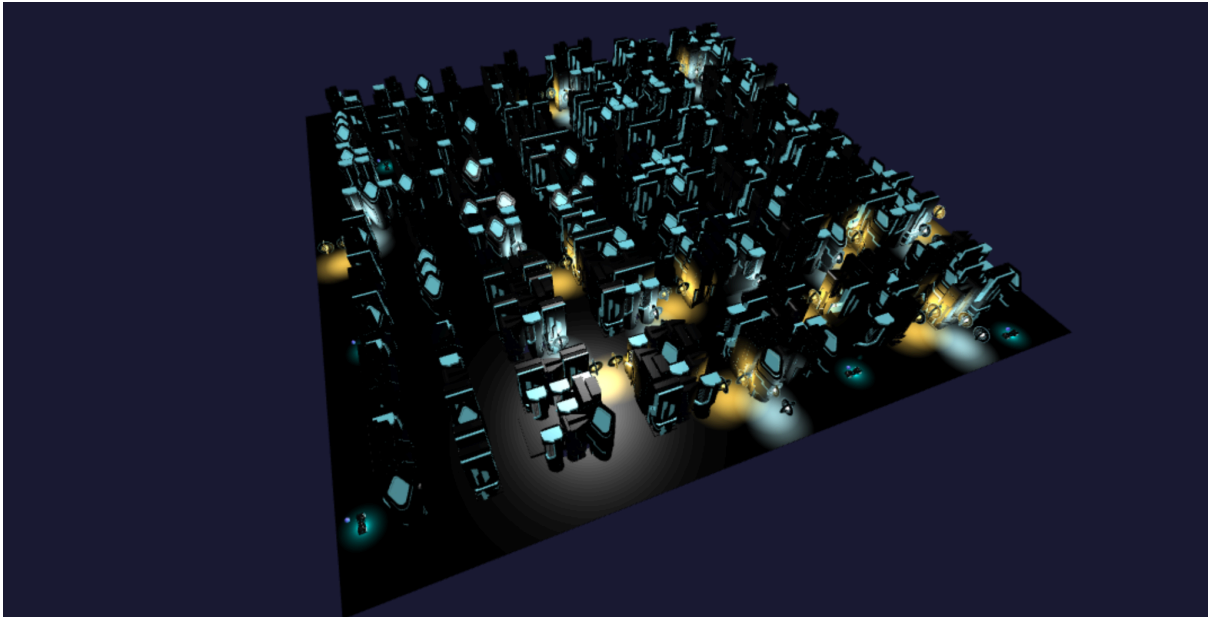


Imagen 1. Mapa de la ciudad, con edificios generados aleatoriamente en base a varios modelos disponibles.

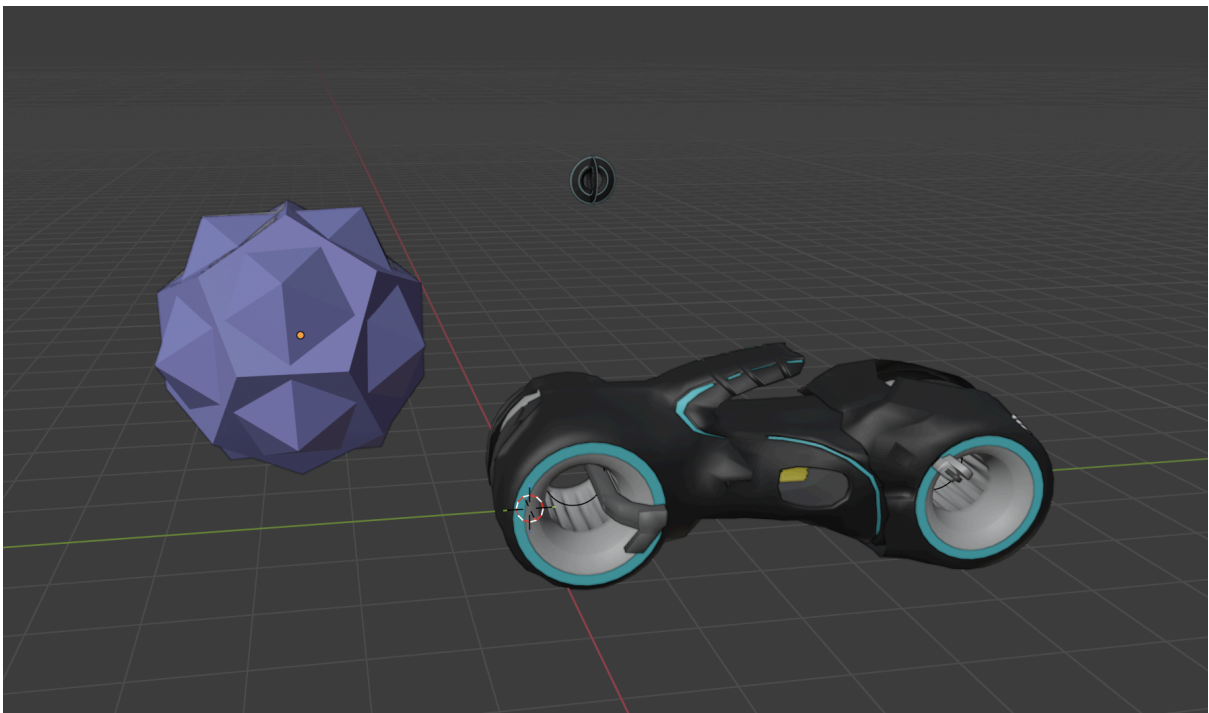


Imagen 2. Modelos utilizados para vehículos y semáforos.

Agentes

- Car (Carros)
 - Objetivo:
 - Llegar a su destino
 - Capacidad efectora:

- Moverse (move()) a la siguiente celda en su ruta planeada
- Detenerse (stop()) en su posición actual cuando no puede avanzar
- Calcular ruta (calculate_route()), usando A* para encontrar el camino óptimo al destino
- Cambiar de carril (can_change_lane()) cuando está bloqueado
- Desaparecer (die()) del modelo al llegar al destino
- Recalcular ruta cuando encuentra obstáculos o tráfico
- Percepción
 - Percepciones Locales (celda actual y adyacentes):
 - Semáforos (has_traffic_light()): Detecta si hay un semáforo y su estado (rojo/verde)
 - Otros carros (can_move_to_cell()): Ve carros bloqueando su camino
 - Tipo de calle (has_road()): Ver si puede pasar por una celda y obtiene su dirección
 - Obstáculos (is_traversable()): Ve si hay obstáculos
 - Percepciones de Rango Medio:
 - Ve proximidad a semáforos (is_near_traffic_light(distance)) en su ruta (1-2 celdas)
 - Ve carriles paralelos (can_change_lane()) para cambiar cuando sea necesario
 - Historial de últimas 5 posiciones (position_history) para evitar loops
 - Percepciones Globales (a través del modelo):
 - Costos de toda la ciudad para hacer rutas (model.edge_costs)
 - Mapa completo de la ciudad (model.grid)
 - Estados de otros carros en intersecciones (get_cars_at_same_intersection())
- Proactividad
 - Calculan rutas óptimas
 - Busca rutas alternativas cuando esta en tráfico
 - Cambiar de carril para distribuir tráfico en cada calle
 - Tiene varios estados para tomar decisiones
- Métricas de desempeño
 - Si llega a su destino o se queda atorado en alguna parte

Arquitectura de Subsunción

- **Capa 0** - Llegar al destino
- **Capa 1** - Salir de semáforos (estando literalmente en la celda del semáforo) si se queda atorado
- **Capa 2** - Calcular ruta
- **Capa 3** - Seguir ruta
- **Capa 4** - Reacción a otros carros o tráfico
 - Recalcular ruta
 - Cambiar de carril
- **Capa 5** - Esperar semáforos
 - Cambiar de carril si hay fila en semáforo para utilizar mejor la calle

Características del Ambiente

El ambiente está conformado por semáforos, obstáculos y calles. Tiene las siguientes propiedades:

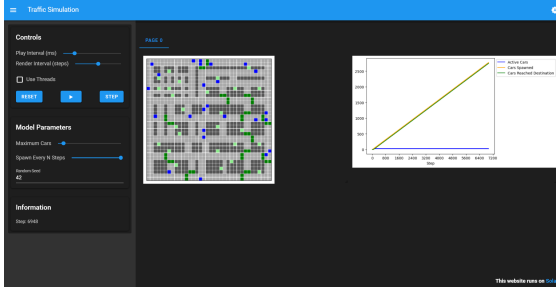
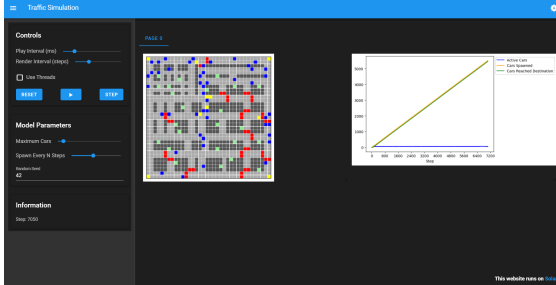
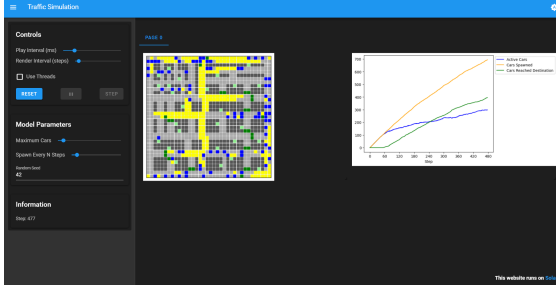
- **Parcialmente accesible**
 - Los carros tienen mucha información, pero les faltan ciertas cosas, como intenciones futuras de otros carros, el estado interno de otros carros (rutas, destinos), los carros que están a más de 2 celdas de distancia (excepto en el cálculo de la ruta).
- **No determinístico**
 - Los destinos son puertos de manera random, esto y otras cosas hacen que si empiezas la simulación con los mismos parámetros, puedas tener varios resultados.
- **No episódico (secuencial)**
 - Los carros si tienen que tomar en cuenta el futuro al calcular rutas, si ven que se genera mucho tráfico en el área al que están yendo, cambian de ruta.
- **Dinámico**
 - Los carros no tienen control sobre los semáforos o direcciones de las calles, pero si son afectados por los mismos.
- **Discreto**
 - Esta simulación tiene un espacio finito, con acciones finitas.

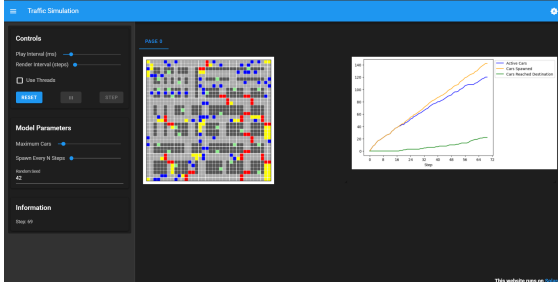
Casos de Prueba

Ahora haremos unos casos de prueba con las siguientes condiciones:

- Carros aparecen cada 10 pasos

- Carros aparecen cada 5 pasos
- Carros aparecen cada 2 pasos
- Carros aparecen cada paso

Pasos	Simulación	Descripción
10		Con esta simulación no hay problema, los carros logran llegar a su destino y parece que puede seguir por mucho tiempo incluso estando por encima de 5000 pasos
5		Al igual que con 10 pasos, parece que está estable y puede seguir por mucho más tiempo
2		En 2 pasos, empieza a ver un problema donde incluso si una esquina puede seguir generando carros, o mínimo parece que puede, la simulación para. Pero también se puede ver que ya se empieza a generar tráfico, principalmente por una calle vertical que atraviesa las 2 glorietsas que hay en el mapa, como que eso restringe las opciones de los demás carros.

1		El mismo problema de 2 pasos ocurre aquí, solo que más rápido, ya que solo aguanta de 50-70 pasos.
---	---	--

Código de Conexión entre Mesa y WebGL

Para lograr que se pudiera ver en 3D el comportamiento de todos los agentes, ya con modelos y con diseños de la temática, el sistema se dividió en dos módulos muy importantes: la parte Lógica (backend en Python) y la Visualización Gráfica (frontend en JavaScript/WebGL). La comunicación entre ambos lados se hace mediante una API.

El intercambio de información se genera mediante un servidor implementado con Flask. Este servidor es el intermedio que pasa los datos de las simulación de Mesa a formato JSON para que sean recibidos por la aplicación gráfica. El flujo de datos es el siguiente:

1. **En el frontend:** se solicita avanzar un step en el tiempo.
2. **En el backend:** se ejecuta la lógica de los agentes y actualiza sus coordenadas.
3. **En el frontend:** se solicitan nuevas posiciones de los agentes y el estado de los semáforos.
4. **En el backend:** se pasa de Python a JSON y responde.
5. **En el frontend:** se actualiza todo lo 3D interpolando las nuevas posiciones.

El archivo `flask_traffic_server.py` inicializa el modelo `CityModel` y define los endpoints. Se usa la librería `flask_cors` para permitir las solicitudes de origen cruzado, que son importantes para la comunicación local entre el servidor y el navegador. Las rutas principales que se implementan son:

- `POST /init`: que recibe parámetros de configuración e instancia la clase `CityModel`. Reinicia la simulación desde cero.
- `GET /getAgents`: recorre la lista de agentes tipo car en el modelo y extrae sus coordenadas, y su ID único. Retorna una lista de diccionarios JSON.
- `GET /getTrafficLights`: este endpoint regresa la posición y el estado de cada semáforo, lo que permite que la visualización cambie el color de las luces y esferas emisoras directamente.

En el lado de `api_connection.js`, se usa `Fetch API` de JavaScript para realizar solicitudes, esto evita que la interfaz gráfica se congele mientras espera una

respuesta del servidor. Para el manejo de agentes el script verifica qué agentes existen en el servidor y sincroniza la lista local. Si un vehículo llega al destino y es eliminado en Python, el script de JavaScript detecta que ese agente falta y lo elimina de la escena 3D.

DrawScene en traffic_Agents.js se encarga de actualizar llamando a la función update() cuando ha transcurrido el tiempo de animación predefinido.

Conclusiones

En conclusión, este proyecto nos mostró mucho acerca de cómo hacer simulaciones de situaciones reales, al hacer esta simulación de una ciudad con carros, nos dimos cuenta de que tanto detalle se le tienen que poner a los agentes y al ambiente para poder generar una simulación parecida a la realidad. Obviamente entendemos que esta simulación fue básica o sencilla, ya que faltan peatones, otros tipos de calles y muchas cosas más que hacen que una ciudad con su tráfico sea tan compleja, pero creemos que esta simulación es una buena introducción al mundo de las simulaciones utilizando gráficas computacionales y agentes.

Algo que notamos en nuestra simulación en específico, es que ocurre un error donde la simulación se acaba cuando 3 o menos esquinas están llenas. Esto puede significar que el carro que acaba de aparecer no se ha movido, o que nuestra manera de decir cuando acaba la simulación necesita una revisión, por lo que sentimos que esa sería la principal mejora que le tendríamos que hacer a la simulación en el futuro, aunque también sería bueno mejorar la lógica de los carros para que no generen tanto tráfico, ya que cuando aparecían los carros cada 2 steps, se empezaba a generar mucho tráfico en la parte de arriba, esto se podría hacer mejorando la distribución de direcciones de las calles, o haciendo que los carros puedan predecir y reaccionar mejor al tráfico.

Video

https://drive.google.com/file/d/1m0UMeV0m6Rdu_CSuiARSBgbyNYxThM8L/view?usp=sharing